CS229S Project 1 实验报告：Transformer 内核、IO 模型与 RTX 5090 上的实测

ShaneLiu04*

2026 年 10 月

摘要

本文对应 Stanford CS229S（Fall 2023）Project 1。原有实验是 CPU 上的 numpy 版本，包括分块 FlashAttention 的前向与反向、HBM 流量（IO）模型、depthwise 因果卷积的几种写法，以及一个权重全部手工设定的两层 associative recall transformer。CPU 上能验证算法对不对，却回答不了「在真实 GPU 上快多少、为什么快」。所以这次在 RTX 5090（sm_120，170 SM，32 GB GDDR7，96 MB L2）上补齐了 GPU 部分，共四件事。第一，用 Triton 写 FlashAttention-2 的前向和反向，与两类基线对比：一类是显式物化 S, P 的朴素 PyTorch 实现，另一类是 `scaled_dot_product_attention` 的 flash、efficient、math 三个后端；扫描范围是 $T = 256 \ldots 32768$、$d \in \{64, 128\}$、因果与非因果、fp16 与 bf16。第二，实测 copy 带宽（1.52 TB/s）和 BF16 矩阵乘峰值（236.3 TFLOP/s），再把 Llama-2-7B 一层的 QKV 投影、attention、输出投影和 MLP 在 prefill 与 decode 形状下画到 roofline 上。第三，写三个 depthwise 1D 卷积 CUDA kernel（一线程一输出、共享内存加 halo、寄存器分块），与 cuDNN `conv1d(groups=C)` 和 FFT 卷积对比，核长从 3 扫到 8192。第四，在 GPU 上训练小 GPT 解 MQAR，并测 tokens/s 与 MFU。

主要发现如下。Triton 前向在长序列上与 PyTorch 自带的 FlashAttention-2 持平（fp16、$d = 64$、$T = 16384$ 时 224.0 对 217.9 TFLOP/s），比朴素实现快 7.15 倍（$T = 8192$ 非因果）到 22.2 倍（因果）。朴素实现与 math 后端在 $B \cdot H = 32$、$T = 16384$ 时 OOM，而 flash 类方法到 $T = 32768$ 时额外显存也只有 138 MB。朴素实现的耗时有 93% 可以用一个简单的式子解释：eager 实现做 $6T^2$ 次 fp16 读写，除以 copy 带宽。反过来，按 IO 模型计算 FlashAttention 的「隐含带宽」，最高能到 6.57 TB/s，是 DRAM 带宽的 4.3 倍。这说明 K/V 的重复读取大部分命中了 96 MB 的 L2，教科书的 $\Theta(T^2 d^2 / M)$ 只是 DRAM 流量的上界。卷积方面，寄存器分块 kernel 在 $K = 1024$ 时达到 45.0 TFLOP/s（按直接卷积的 FLOPs 折算），比 cuDNN 快 6.5 倍，在 $N = 16384$ 时仍快于 FFT；短卷积 $K = 4$ 时它跑到 copy 带宽的 97%。训练方面，2 层 GPT 在 MQAR 上于第 1000–1100 步从 0.17 的平台突然跳到 100%，1 层模型始终学不会；用本文 Triton kernel（含反向）训练的学习曲线与 SDPA 重合。在 GPT-2-small 形状下，bf16 加 SDPA 达到 159.6k tokens/s、MFU 0.447，是 fp32 朴素实现的 3.84 倍。

目录

1 背景与作业要求 **3**

2 实验环境 **3**

*GitHub: `https://github.com/ShaneLiu04`。代码与原始指标：`https://gitee.com/liu-xingyan04/mlsys-infra`。本文基于公开课程材料独立完成，与课程团队无隶属关系。

3 方法与实现 **4**
3.1 Triton FlashAttention-2 前向 4
3.2 Triton 反向 5
3.3 计时与正确性方法 5
3.4 Roofline 与一层 Llama-2-7B 5
3.5 Depthwise 卷积的三个 CUDA kernel 5
3.6 训练 6

4 实验结果 **6**
4.1 CPU 部分回顾 6
4.2 FlashAttention 前向：时间、吞吐、显存与误差 8
4.3 反向 10
4.4 batch×heads 与占用率 11
4.5 块大小扫描与 IO 模型 11
4.6 Roofline：一层 Llama-2-7B 的 prefill 与 decode 13
4.7 Depthwise 卷积 15
4.8 训练：MQAR 与吞吐 17
4.9 手写 associative recall 在 GPU 上的复核 19

5 与理论模型的对照 **20**
5.1 Attention 的 FLOPs、字节与强度 20
5.2 为什么 FlashAttention 快：三个因素的分解 20
5.3 Roofline 中的线性层 20
5.4 卷积 21

6 面试专题 **21**

7 局限 **23**

8 结论 **24**

1 背景与作业要求

CS229S Project 1 由四个官方部分组成，表 1 列出了它们与本文各节的对应关系。FlashAttention 不是官方必做项，但第 3 部分要求「分析多头注意力的算术强度」，要把这个分析落到实测上，就需要一个能测的 attention kernel，所以我把它作为 GPU 部分的主线。

表 1: 官方作业与本文实现的对应关系

官方部分	本文实现	位置
1. 训练一个小 Transformer	GPU 上训练 1 层和 2 层 GPT 解 MQAR（多查询 associative recall），比较 bf16 autocast、SDPA flash、朴素 attention 和自写的 Triton attention；在 GPT-2-small 形状下测 tokens/s 与 MFU	第 4.8 节
2. 手写两层 Transformer 解 associative recall	CPU 部分：上一 token 头加匹配头，权重全部手工设定；这次在 GPU 上复核，24 组配置与 CPU 的预测完全一致	第 4.1 节 and 4.9 节
3. MHA 的算术强度	CPU 部分：朴素与 flash 的 HBM 字节模型，以及 decode 阶段 MHA、GQA、MQA 的强度；GPU 部分：实测 roofline，看 Llama-2-7B 一层四个算子在 prefill 与 decode 形状下落在哪里	第 4.6 节 and 5 节
4. 1D depthwise 卷积 CUDA kernel	CPU 部分：numpy 的四种写法；GPU 部分：naive、共享内存加 halo、寄存器分块三个 CUDA kernel，对比 cuDNN 与 FFT	第 4.7 节
（补充） FlashAttention	CPU：numpy 分块前向、反向和 IO 计数器；GPU：Triton 前向与反向，以及块大小扫描	第 4.2 节

2 实验环境

性能结论离不开硬件参数，尤其是 roofline 的两个「屋顶」：显存带宽和矩阵乘峰值。下表列出软硬件环境，其中最后一行是我自己测出来的峰值，后文所有比例都以它为分母。

表 2: 软硬件环境（GPU 信息来自 `attention_fwd.jsonl` 的 meta 行，CPU 信息来自 `meta.json`）

GPU	NVIDIA GeForce RTX 5090，compute capability 12.0（sm_120），170 SM，33.67 GB 可见显存，96 MB L2；规格为 GDDR7 1.792 TB/s，dense BF16 张量核 209.5 TFLOP/s（按 boost 时钟）
GPU 软件	Python 3.12.3，PyTorch 2.8.0+cu128，Triton 3.4.0，CUDA 12.8（nvcc `-arch=sm_120`），cuDNN 9.10.2
GPU 主机	AutoDL 容器，7 个任务共享；所有 GPU 计时都在共享锁 `flock gpu.lock` 内独占 GPU 完成，CPU 绑定在 2 个核上（`taskset -c 38,40`）
CPU 部分	Windows Server，Intel Ice Lake（Family 6 Model 106），4 个物理核、8 个逻辑核，与其他任务共享；Python 3.12.10，numpy 2.5.3，torch 2.6.0 CPU 版
实测峰值	copy 带宽 1521.9 GB/s（2 GiB 缓冲），只读求和 1625.6 GB/s；BF16 `matmul` 峰值 236.3 TFLOP/s（$n = 6144$），fp16 220.3，TF32 113.8，fp32 65.7；ridge point 为 $236.3/1.522 = 155.3$ FLOP/B

表里有一个反常的地方：实测 BF16 峰值比规格还高 13%。原因在时钟。规格按 2.41 GHz 的 boost 时钟计算，而这块卡的最高 SM 时钟是 3090 MHz（`nvidia-smi` 的 `clocks.max.sm`），持续负载下实际时钟高于标称 boost。因此本文 roofline 与 MFU 的分母一律用实测值（236.3 TFLOP/s、1.522 TB/s），图中同时画出规格屋顶作参照。

另一个限制会贯穿全文：在这个容器里 ncu 报 `ERR_NVGPUCTRPERM`，也就是没有权限读性能计数器（记录在 `ncu_attention.jsonl`）。所以我拿不到 DRAM 字节数的直接测量，只能用「模型字节数除以实测时间」得到一个隐含带宽，再从它推断真实流量（第 4.5 节）。

3 方法与实现

这一节说明几个 kernel 是怎么写的、怎么计时、怎么判断结果对不对。读者如果只关心结论，可以先跳到实验结果，遇到不明白的实现细节再回来查。

3.1 Triton FlashAttention-2 前向

算法来自 FlashAttention[1] 与 FlashAttention-2[2]，外层循环沿 Q 块走；online softmax（边扫描边更新最大值和归一化因子的 softmax）来自 [3, 4]；用 Triton[6] 实现。每个 Triton program 负责一个 $(Q$ 块$, b \cdot h)$ 组合。它先把 B_r 行 Q 读进片上存储，然后沿 K/V 方向以 B_c 为步长循环，一路维护三样东西：每行的最大值 m、归一化因子 ℓ，以及还没除以 ℓ 的输出 acc。实现里有三个值得交代的细节：

1. 以 2 为底的 online softmax。把 $\log_2 e$ 并入缩放系数后，就可以用 `exp2` 代替 `exp`，因为 GPU 上 `ex2.approx` 是一条 SFU（特殊函数单元）指令。最后存下 $\mathrm{lse} = (m + \log_2 \ell) \ln 2$ 供反向使用。
2. 因果掩码分两段处理。对角线左侧的整块不需要掩码；只有跨过对角线的块才做 `tl.where`；对角线右上方的块根本不加载。对角段的起点取 $\lfloor sB_r/B_c \rfloor B_c$，这样任意 B_r, B_c 组合都是对的，测试里也覆盖了 $B_r < B_c$ 的情况。
3. 反向的调度顺序。因果时让 program 0 处理工作量最大的最后一个 Q 块，以减少尾部的负载不均。

下面是前向内层循环的核心代码。每一轮读一块 K，算出分数，更新最大值并把旧状态按比例缩放，再乘上 V 累加：

Listing 1: 前向内层循环（`gpu/flash_triton.py`，有删节）

```
for start_n in range(lo, hi, BLOCK_N):
    cols = start_n + offs_n
    k = tl.load(k_base + cols[None, :] * HEAD_DIM + offs_d[:, None]) # (D, Bc)
    qk = tl.dot(q, k) * qk_scale # qk_scale = sm_scale * log2(e)
    if MASK_CAUSAL:
        qk = tl.where(offs_m[:, None] >= cols[None, :], qk, float("-inf"))
    m_new = tl.maximum(m_i, tl.max(qk, 1))
    alpha = tl.math.exp2(m_i - m_new) # rescale old state
    p = tl.math.exp2(qk - m_new[:, None])
    l_i = l_i * alpha + tl.sum(p, 1)
    v = tl.load(v_base + cols[:, None] * HEAD_DIM + offs_d[None, :]) # (Bc, D)
    acc = tl.dot(p.to(v.dtype), v, acc * alpha[:, None])
    m_i = m_new
```

默认配置按顺序尝试，直到共享内存放得下为止：$d = 64$ 用 $(B_r, B_c, \text{warps}, \text{stages}) = (128, 64, 4, 3)$，$d = 128$ 用 $(128, 64, 8, 2)$。之所以 $d = 128$ 只能用两级流水，是因为 sm_120 每个 block 最多只有 99 KB 共享内存，比 H100 的 227 KB 小得多。$d = 128$ 时 Q 块占 32 KB，两级流水的 K/V 共 64 KB，刚好放下，三级就放不下了。

3.2 Triton 反向

反向与 CPU 版 `flash_backward` 的结构相同。先用 PyTorch 算 $D_i = \sum_j dO_{ij} O_{ij}$，然后启动两个 kernel：`_attn_bwd_dkdv` 的每个 program 持有一个 K/V 块，沿 Q 方向循环，重算 $P^\top = \exp(KQ^\top \cdot s - \text{lse})$，并累加 $dV \mathrel{+}= P^\top dO$ 和 $dK \mathrel{+}= dS^\top Q$；`_attn_bwd_dq` 的每个 program 持有一个 Q 块，沿 K/V 方向循环累加 $dQ \mathrel{+}= dS\,K$。两个 kernel 都不写出 P，也不需要原子操作。这样写简单，但有代价：$QK^\top$ 和 $dO\,V^\top$ 各被重算了两次。FA2 原文用一个 kernel 加原子累加 dQ，Triton 官方教程则在一个 kernel 里同时做两件事。所以我的反向比 PyTorch 的 FA2 慢约 20%，第 4.3 节 会定量解释这个差距。

3.3 计时与正确性方法

- 计时：用 CUDA event，先预热 2–5 次，再取 10–30 次的中位数；单次测量时长上限 1.5 s。
- 峰值显存：调用前执行 `reset_peak_memory_stats`，报告 `max_memory_allocated` 减去调用前的已分配量，也就是「输入之外的额外显存」。OOM 会被捕获并记录，之后更长的 T 直接标为 `skipped_after_oom`。
- 误差：对 batch 0、head 0，用 fp32 的分块参考实现（按 query 分块，显存 $O(2048 \cdot T)$）计算 $\max |O - O_{\text{fp32}}|$；反向梯度对前 2 个 head 用 fp32 的 math 后端 autograd 作参考（$T \le 4096$）。
- FLOPs：前向按 $4BHT^2d$ 计，因果时只计有用的一半；前向加反向按 FA2 论文记为前向的 3.5 倍，因为反向有 5 个矩阵乘，前向有 2 个。

3.4 Roofline 与一层 Llama-2-7B

Roofline 模型 [5] 需要两个参数。copy 带宽用 `y.copy_(x)`（一读一写）在 64 KiB 到 2 GiB 之间扫描，峰值取缓冲 $\ge$ 256 MiB（远超 96 MB L2）时的最大值；BF16 峰值取方阵乘 $n = 256 \ldots 16384$ 中的最大值。一层的四个算子按 Llama-2-7B[16] 的形状设置（$d = 4096$，32 头，$d_h = 128$，$d_{ff} = 11008$，gated MLP），精度为 bf16：QKV 投影是 $[M, d] \times [d, 3d]$；attention 用 SDPA 的 flash 或 efficient 后端；输出投影是 $[M, d] \times [d, d]$；MLP 是 $[M, d] \times [d, 2d_{ff}]$、`silu`·、$[M, d_{ff}] \times [d_{ff}, d]$。FLOPs 与字节一律由 `tools/mlsys_calc.py` 的 `gemm_intensity`（假设每个矩阵只过一次 HBM）和 `kv_cache_bytes` 计算。因此图上的横坐标是解析出来的算术强度（每读一个字节做多少次浮点运算），纵坐标是实测吞吐。prefill 取 $B = 1$、$T \in \{128, 512, 2048, 8192\}$ 以及 $B = 8, T = 2048$；decode 取上下文 4096、$B \in \{1, 4, 16, 64, 256\}$。

3.5 Depthwise 卷积的三个 CUDA kernel

要算的是 $y[b, c, t] = \sum_{j<K} h[c, j]\, x[b, c, t - j]$，即每个通道各自用自己的滤波器做因果卷积。H3、Hyena[7, 8] 与 Mamba[9] 里的短卷积和长卷积都是这个算子。精度是 fp32，用 `torch.utils.cpp_extension.load_inline` 编译（`-O3 -arch=sm_120`）。三个版本一步步减少对慢速存储的访问：

- naive：一个线程算一个输出，做 K 次全局内存读取 x 和 h，数据复用全靠 L1/L2 缓存。
- smem：一个 block 负责一行 (b, c) 的 256 个输出，先把 $x[t_0 - K + 1,\, t_0 + 256)$（包含长度 $K - 1$ 的 halo，即块边界外需要的历史数据）和 $h[c, :]$ 搬进共享内存。但每个 FMA 仍要读 2 次共享内存（h_j 是广播读，x 是连续地址，没有 bank 冲突）。
- regblock：每个线程负责 $R = 8$ 个连续输出，抽头按 R 个一组处理。先把 $2R - 1$ 个 x 读进寄存器窗口 w，再做 $R \times R$ 个 FMA，所以 64 个 FMA 只需要 23 次共享内存读。线程 i 读 `sx[8i+c]`

本来会产生 8 路 bank 冲突，我用填充下标 `PADI(i)=i+(i>>5)` 消除了它；输出先写回共享内存，再合并写到全局内存。

寄存器分块的核心循环如下，内层两重循环完全展开，w 和 `acc` 都留在寄存器里：

Listing 2: 寄存器分块 kernel 的核心循环（`gpu/dwconv_cuda.py`）

```
for (int jb = 0; jb < Kp; jb += R) { // taps in chunks of R (h zero-padded to Kp)
  const int base = o0 + Kp - jb - R;
  float w[2 * R - 1];
#pragma unroll
  for (int i = 0; i < 2 * R - 1; ++i) w[i] = sx[PADI(base + i)];
#pragma unroll
  for (int jj = 0; jj < R; ++jj) {
    const float hv = sh[jb + jj]; // broadcast
#pragma unroll
    for (int r = 0; r < R; ++r) acc[r] = fmaf(hv, w[r - jj + R - 1], acc[r]);
  }
}
```

3.6 训练

`gpu/tiny_gpt.py` 是一个 pre-LN GPT，使用可学习的位置编码，输入和输出嵌入共享权重。attention 实现可以切换为 `naive`、`sdpa`（自动选 flash）或 `triton`（本文的 kernel，前向加反向）。

训练任务是 Zoology[10] 提出的 MQAR。每条序列形如 $[k_1 v_1 \cdots k_P v_P \,|\, q_1 a_1 \cdots q_P a_P]$：前半段给出 P 个键值对，后半段按随机顺序查询这些 key，模型要答出对应的 value。P 个 key 从 n_k 个 token 中无放回抽取，value 来自另外 n_k 个 token，只在答案位置计算 loss。模型宽度 $d = 128$、4 头，batch 256，每步重新采样，所以没有重复数据；优化器是 AdamW（$\beta = (0.9, 0.98)$，100 步 warmup 加余弦衰减，梯度裁剪 1.0）。我先在 $(n_k, P, \text{lr}, \text{wd})$ 上扫描了两轮（2000 步一轮、6000 步一轮），再取「eval 准确率 ≥ 0.9 的设置里最难的那个」跑 6000 步对照实验（`chosen_mqar_config`）。

吞吐实验换成 GPT-2-small 形状（12 层、$d = 768$、12 头、$T = 1024$、词表 32768、$B = 8$），比较 fp32、TF32、bf16 autocast 三种精度与三种 attention。MFU（模型 FLOPs 利用率，即有用 FLOPs 占峰值的比例）用 `mlsys_calc.mfu` 计算，公式是 $6N \cdot \text{tokens/s}/\text{peak}$；同时报告计入 attention 的 $6N + 12LTd$ 版本（PaLM 附录 B[12]）。

4 实验结果

4.1 CPU 部分回顾

在看 GPU 数字之前，先回顾 CPU 部分得出的三个「理论预期」，后面的 GPU 实验会逐一检验它们。CPU 部分的完整数字见实验目录的 README。

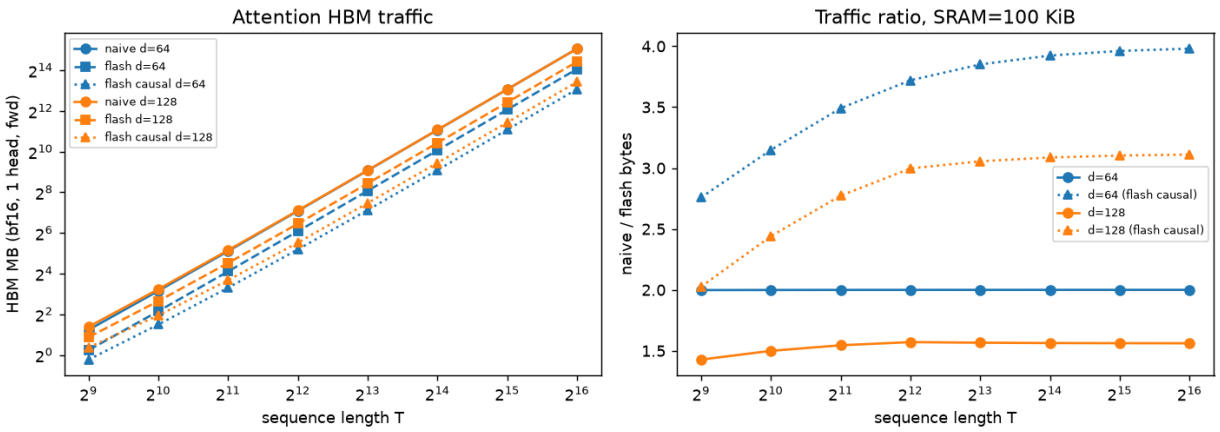


图 1: CPU 部分的 IO 模型（`attention_io.csv`）：SRAM=100 KiB、bf16、单头前向。左：朴素与 flash 的 HBM 字节随 T 的变化；右：二者之比。

第一个预期来自图 1。按 FlashAttention 论文的块大小 $B_c = \lfloor M/4d \rfloor$、$B_r = \min(B_c, d)$，$d = 64$ 时 $B_r = 64$。每个 Q 块都要把 K/V 整段读一遍，一共重读 T/B_r 遍，所以 flash 的流量约是 $2T^2 d/B_r \approx 2T^2$ 个元素，而朴素实现是 $4T^2$。两者之比因此固定在 2.00；因果时约 4 倍（$T = 65536$ 时 3.98）。这张图暗示：FlashAttention 的主要收益不在于改变流量的阶，而在于把显存从 $O(T^2)$ 降到 $O(T)$。GPU 结果会修正这句话的后半部分。在真实 GPU 上，朴素实现的流量常数比 4 大（eager 实现是 6 或 8），flash 的块也比 d 大（$B_r = 128$），而 L2 又吸收了大部分 K/V 重读，所以时间上的差距远不止 2 倍。

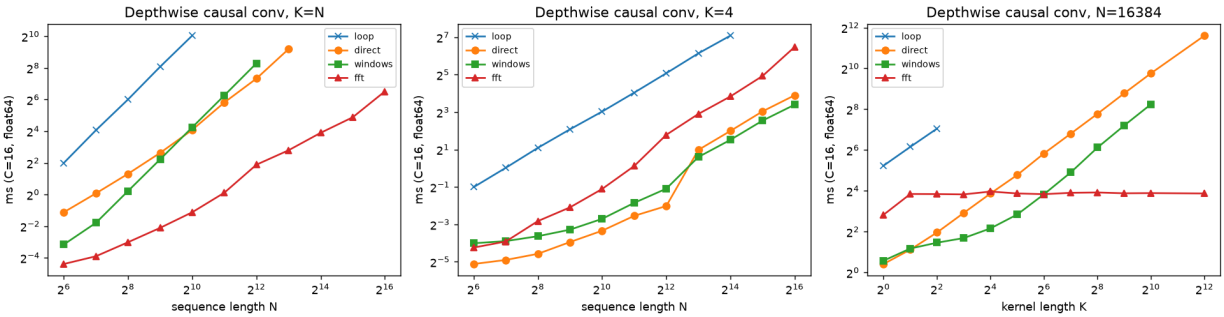


图 2: CPU 部分的 depthwise 因果卷积计时（`conv1d_timing.csv`，$C = 16$，float64，numpy）。

第二个预期是卷积何时该用 FFT。图 2 中 FFT 的耗时只取决于 N（$N = 16384$ 时 14–15 ms），而 $O(NK)$ 的直接写法随 K 线性增长。CPU 上，移位相加在 $K \approx 16$ 处与 FFT 交叉，滑窗在 $K \approx 64$ 处交叉。到了 GPU 上，交叉点会右移到 $K \approx 1600$（第 4.7 节）。原因是直接卷积在 GPU 上能用上 45 TFLOP/s 的算力，而 FFT 受访存限制，吃不到这份算力。

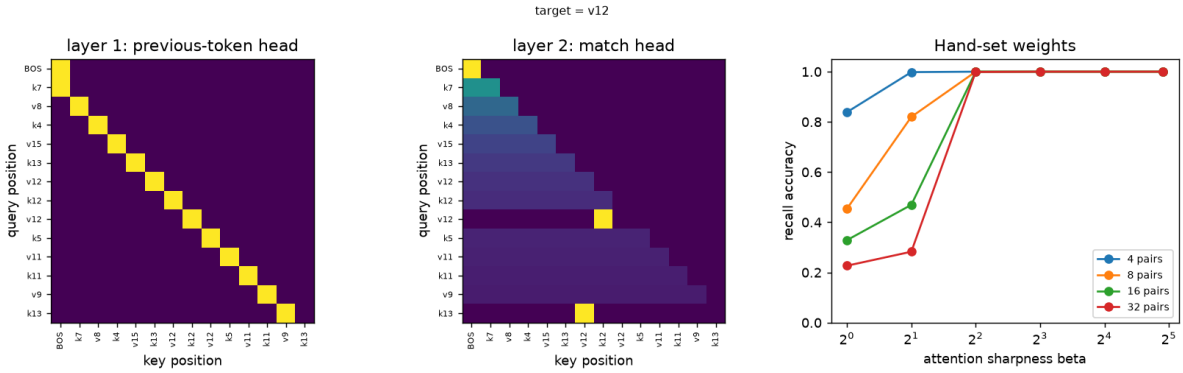


图 3: 手工设定权重的两层 associative recall transformer（`recall_accuracy.csv`）。左、中：两层的注意力图；右：准确率随注意力锐度 β 的变化。

第三个预期是 associative recall 至少需要两层。图 3 展示了手工构造的两步机制。第 1 层的 query 是 $\beta\, e_{\text{pos}}(p)$，key 是 $e_{\text{pos}}(j+1)$，于是位置 p 注意到 $p-1$，把 $\text{tok}(p-1)$ 写进一个叫 prev 的子空间。第 2 层的 query 是 $\beta\,\text{tok}(p)$，key 是 $\text{prev}(j)$，于是最后一个位置（查询键 q）会注意到「前一个 token 是 q」的那个位置，也就是对应的 value。$\beta \geq 8$ 时，4–32 对键值都能 100% 答对；$\beta = 1$ 时注意力太平，32 对只有 0.2268。这正是 induction head[11] 的两步结构。后面的 MQAR 训练会说明，1 层模型确实学不出它（第 4.8 节）。

4.2 FlashAttention 前向：时间、吞吐、显存与误差

这一节回答 GPU 部分最核心的问题：自己写的 FlashAttention 和工业实现相比如何，比朴素实现快多少，省了多少显存，数值上是否可靠。先看时间和吞吐随序列长度的变化。

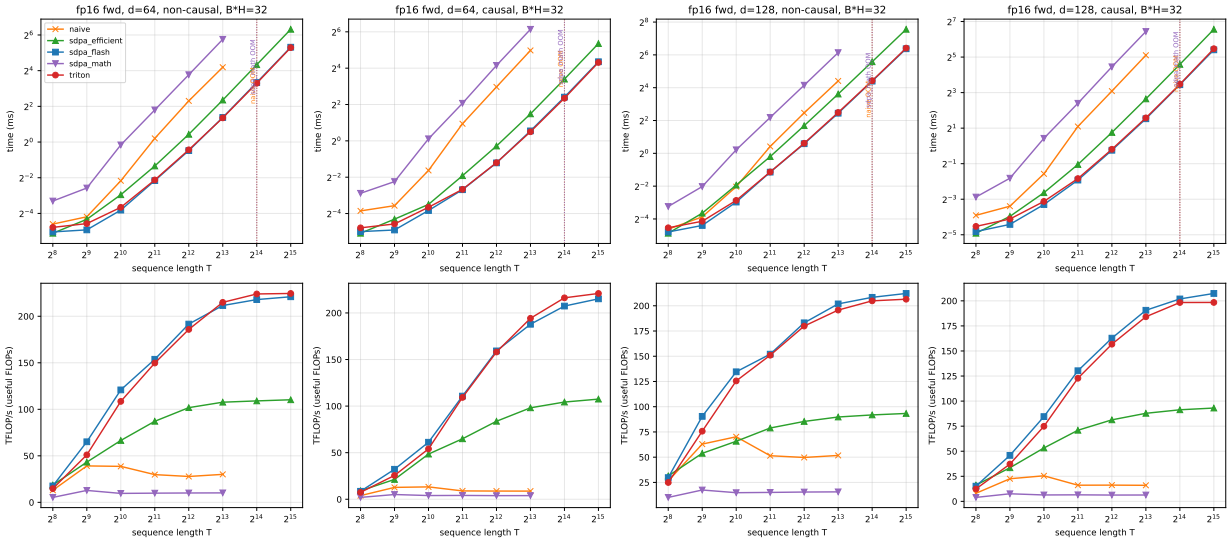


图 4: fp16 前向，$B = 2, H = 16$。上：时间（log-log），竖虚线是 OOM 点；下：按有用 FLOPs 计算的吞吐（`attention_fwd.jsonl`）。

表 3: fp16 前向，$d = 64$，非因果，$B \cdot H = 32$（每格为 ms / TFLOP/s；「额外显存」指输入之外的峰值）

方法	$T = 1024$	$T = 4096$	$T = 8192$	$T = 16384$	$T = 32768$
Triton（本文）	0.079 / 108.5	0.739 / 185.9	2.558 / 214.9	9.818 / 224.0	39.18 / 224.5
SDPA flash	0.071 / 120.9	0.717 / 191.6	2.599 / 211.5	10.09 / 217.9	39.81 / 221.0
SDPA efficient	0.129 / 66.5	1.350 / 101.8	5.107 / 107.6	20.17 / 109.0	79.78 / 110.2
SDPA math	0.894 / 9.6	13.66 / 10.1	54.06 / 10.2	OOM	—
朴素 PyTorch	0.222 / 38.7	4.943 / 27.8	18.28 / 30.1	OOM	—
额外显存：Triton	4.3 MB	17.3 MB	34.6 MB	69.2 MB	138.4 MB
额外显存：朴素	138 MB	2.16 GB	8.62 GB	OOM	—

从图 4 and 表 3 可以读出四点。

本文的 Triton kernel 与 PyTorch 自带的 FlashAttention-2 基本持平。长序列时略快：$d = 64$ 非因果、$T = 16384$ 时是 224.0 对 217.9 TFLOP/s，因果时是 216.2 对 207.4。短序列时略慢，$T = 1024$ 时是 108.5 对 120.9；$d = 128$ 时慢 2–4%（$T = 32768$ 因果为 198.4 对 207.4）。短序列的差距来自启动和尾部效应。$T = 1024$、$B_r = 128$ 时总共只有 $8 \times 32 = 256$ 个 program，平均每个 SM 只分到 1.5 个，最后一波只有一半 SM 在干活，这种「波次量化」损失很明显。

长序列时 attention 已经是计算受限的。峰值 224.5 TFLOP/s 是实测 BF16 方阵乘峰值（236.3）的 95%，几乎没有再提升的空间。

朴素实现则是访存受限的。它的吞吐随 T 增长反而下降，停在 28–30 TFLOP/s（$d = 64$）；因果时只有 8.7 TFLOP/s。第 5 节 会用它的访存量把这个数解释到 93%。因此 $T = 8192$ 时 Triton 比朴素实现快 7.15 倍（非因果），因果时快 22.2 倍（31.47 对 1.415 ms）。

最后，SDPA 的 math 后端比朴素实现还慢 3 倍（$T = 4096$ 时 13.66 对 4.94 ms），额外显存也是朴素的 2.3 倍（4.97 对 2.16 GB）。看起来它在 fp16 输入下仍然走了通用路径，生成了更多中间张量。

时间之外，FlashAttention 更直接的收益是显存，下图把这一点单独画出来。

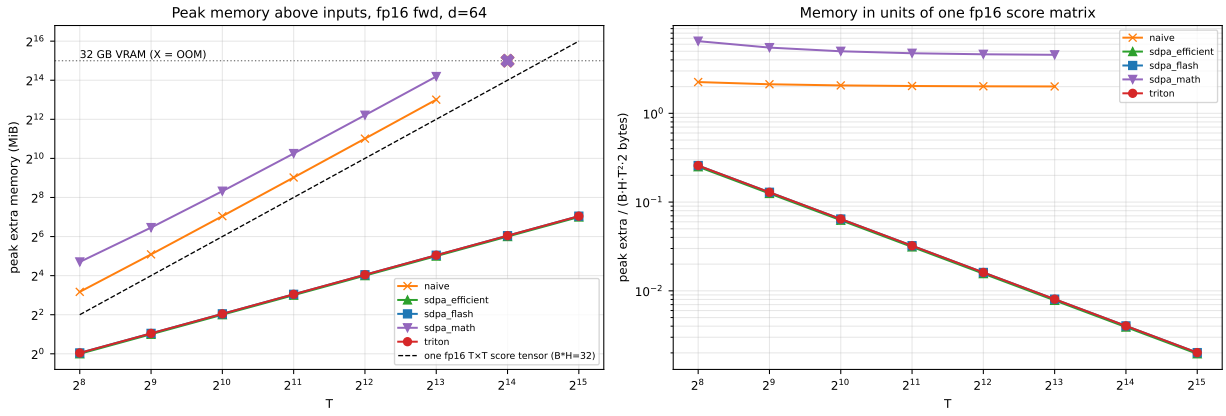


图 5: 峰值额外显存（fp16，$d = 64$，非因果）。左：绝对值，虚线是一个 fp16 $T \times T$ 分数张量；右：以一个分数张量为单位。

图 5 中，朴素实现的峰值恰好是 2 个 fp16 分数矩阵。以 $T = 8192$ 为例，$8.62\ \text{GB} = 2 \times 32 \times 8192^2 \times 2\ \text{B}$。这两个矩阵是 matmul 写出的 S 和 softmax 写出的 P，它们在某一时刻同时存活；中间 `*scale` 产生的临时张量在下一步就被释放了。math 后端则要 4 个以上。有了这个规律，OOM 点就可以预先算出来：32 GB 的卡在 $B \cdot H = 32$ 时，$T = 16384$ 的两个分数矩阵需要 34.4 GB，必然放不下，

这正是实际记录到的 OOM 点。flash 类方法的额外显存只有输出 O 加上 lse，即 $32\times T\times(64\times 2+4)$ 字节，$T=32768$ 时为 138.4 MB。它与 T 成线性关系，所以右图（以分数张量为单位）的斜率是 -1。

速度和显存都占优之后，还要确认数值没有变差。

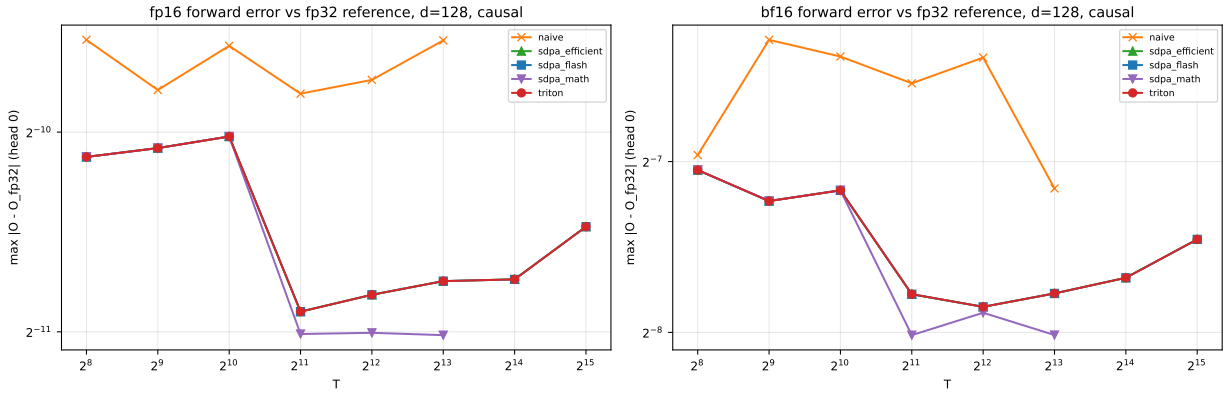


图 6: 与 fp32 参考的最大绝对误差（head 0，$d=128$，因果）。左 fp16，右 bf16。

图 6 中，所有 flash 类实现（Triton、SDPA flash 和 efficient）的误差几乎相同。原因是它们的做法一样：在 fp32 中累加，只在 $P\cdot V$ 之前把 P 舍入为 16 位。以 $T=4096$、非因果、$d=64$ 为例，fp16 的最大误差是 6.6e-5，bf16 是 3.65e-4，比值约 5.5，接近两种格式尾数精度之比（2^{-11} 对 2^{-8}，即 8 倍）。朴素实现的误差大 2–4 倍（fp16 下 1.43e-4），因为它把 S 也以 fp16 写回显存后才做 softmax，多了一次舍入。

图中还有一个现象：因果时误差反而更大（fp16 约 5e-4–9e-4），而且不随 T 下降。原因是因果序列的前几行只对很少几个 value 取平均，舍入误差没有被平均掉。非因果时误差随 T 下降（$T=32768$ 时 2.2e-5），因为每个输出都是对更多 value 的平均，幅值本身变小了。

4.3 反向

训练需要反向。这里要回答两个问题：Triton 反向算得对不对，以及两个独立 kernel 的结构要付出多少速度代价。

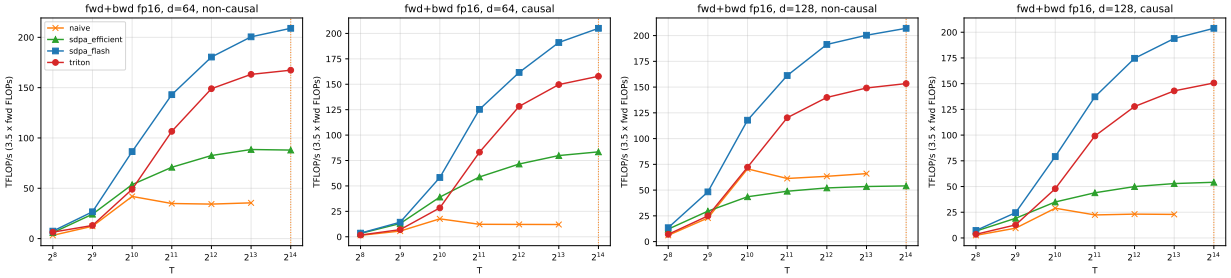


图 7: 前向加反向的吞吐（fp16，$B\cdot H=32$，FLOPs 记为前向的 3.5 倍；`attention_bwd.jsonl`）。

正确性没有问题：在 $T=4096$、$d=64$、非因果时，Triton 与 SDPA flash 的 dQ 最大误差都是 8.8e-5。速度上，图 7 显示 Triton 是 SDPA flash 的 80%。$d=64$ 非因果 $T=16384$ 时耗时 45.99 对 36.83 ms（167.4 对 209.0 TFLOP/s）；$d=128$ 时为 153.4 对 207.1 TFLOP/s。

这个 80% 可以从结构上算出来。如第 3.2 节 所述，两个独立 kernel 各自重算 $QK^\top$ 与 $dOV^\top$，所以反向有 7 个矩阵乘而不是 5 个。加上前向的 2 个，总量是 9 对 7。如果每个矩阵乘的效率相同，

我的实现最多只能达到 $7/9 = 78\%$。实测 80% 说明单个矩阵乘的效率与 FA2 相当，差距几乎全部来自多做的那两次计算。

短序列时还有另一种开销。Triton 在 $T = 256$ 到 1024 之间有一个约 0.53 ms、几乎不变的平台，这是主机侧开销：autograd、用 PyTorch 计算 D、两个 Triton launcher，在只有 2 个 CPU 核的环境下各要几十微秒。另外两条基线也值得一提：朴素实现要为反向保存 P，$T = 8192$ 时峰值显存达 17.1 GB，$T = 16384$ 时 OOM；efficient 后端在 $d = 128$ 时只有 54 TFLOP/s。

4.4 batch×heads 与占用率

前面的实验都固定 $B \cdot H = 32$。但 decode 等场景下并行度很小，这时 kernel 能否把 170 个 SM 都用起来就成了问题。

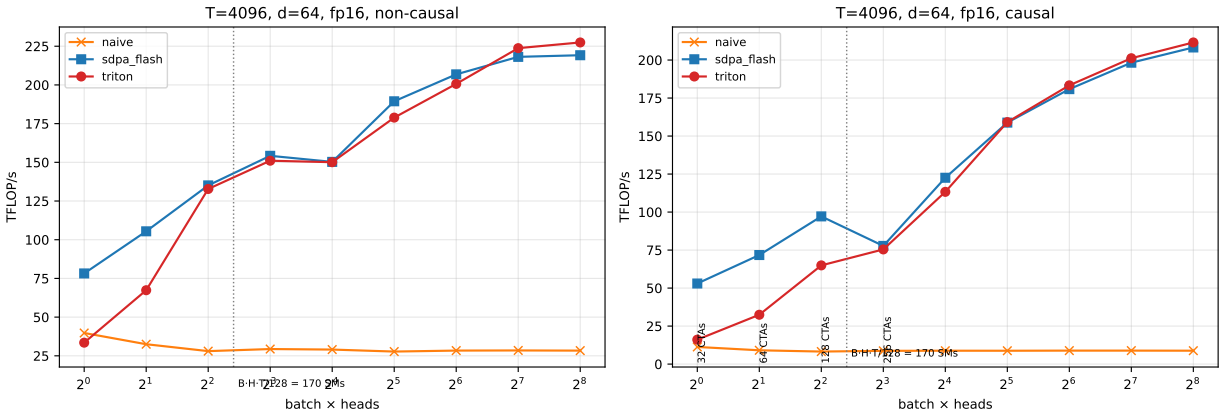


图 8: $T = 4096$、$d = 64$ 时吞吐随 $B \cdot H$ 的变化（`attention_bh.jsonl`）。标注为 Triton 默认块下的 program 数。

Triton 的网格大小是 $\lceil T/B_r \rceil \times BH$。图 8 中，$BH = 1$ 时只有 32 个 program，而 GPU 有 170 个 SM，所以 Triton 只有 33.5 TFLOP/s。SDPA flash 在同样形状下有 78.2 TFLOP/s，说明它对小网格用了更小的块或者拆分策略。$BH \geq 4$（128 个 program）之后两条曲线几乎重合，$BH = 256$ 时 Triton 是 227.4、SDPA flash 是 219.2 TFLOP/s。因果时小网格的劣势更明显（$BH = 1$ 时 16.1 对 53.0），因为各 Q 块的工作量不同，总时间由最重的那个 program 决定。这条曲线也解释了 decode 阶段为什么要用 split-K（Flash-Decoding[15]）：decode 时每个序列只有 1 行 query，网格天然太小，必须沿 K/V 方向再切。

4.5 块大小扫描与 IO 模型

块大小 B_r 在 IO 模型里决定了 K/V 被重读多少遍。如果 IO 模型准确描述了 GPU 上的 DRAM 流量，那么扫描块大小时，耗时应该跟着模型字节数走。这一节就来检验这件事。

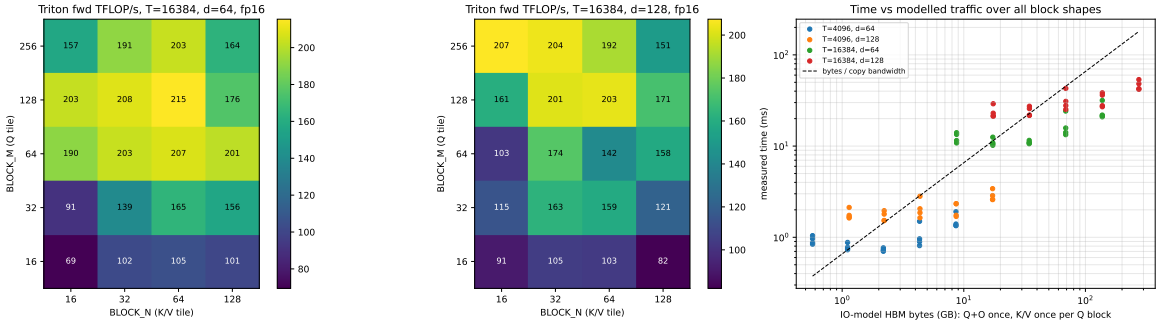


图 9: Triton 前向的 $B_r \times B_c$ 扫描（fp16，非因果，$B \cdot H = 32$，`attention_blocks.jsonl`）。左、中：$T = 16384$ 的 TFLOP/s；右：所有块形状的实测时间对 IO 模型字节，虚线是「字节 ÷ copy 带宽」。

IO 模型沿用 CPU 部分 `flash_elements` 的计数方式：Q 和 O 各读写一次，每个 Q 块把整段 K/V 读一遍，

$$\text{bytes}_{\text{flash}} = BH\left[2\left(2Td + \lceil T/B_r \rceil \cdot 2Td\right) + 4T\right]. \tag{1}$$

先看图 9 的左、中两图。$d = 64$ 时最佳块是 $(B_r, B_c) = (128, 64)$，达到 215.4 TFLOP/s；$d = 128$ 时 $(256, 16)$ 的 207.4 与 $(128, 64)$ 的 202.8 很接近。$B_r = 16$ 时吞吐只有一半（$d = 64$ 时 105.0），$B_r = 256$ 时又下降到 203.2；$B_c = 128$ 普遍较差，原因是寄存器溢出，或者共享内存只够单级流水。

右图是本节的重点。如果式 (1) 就是真实的 DRAM 流量，那么所有点都不可能跑得比 copy 带宽更快，应该落在虚线（1.52 TB/s）上方。实际上 $B_r = 16$ 的点全在虚线下方很远。以 $T = 16384$、$d = 64$、块 $(16, 64)$ 为例，模型字节是 137.6 GB，耗时 20.94 ms，隐含带宽 6.57 TB/s，是 DRAM 带宽的 4.3 倍。唯一合理的解释是大部分 K/V 重读命中了 L2。同一个 head 的 K/V 只有 $2 \times 16384 \times 64 \times 2 = 4$ MB，同时运行的几百个 program 中，属于同一 head 的 Q 块共享这 4 MB；96 MB 的 L2 能放下 24 个 head 的 K/V。所以教科书 IO 模型在 GPU 上只是 DRAM 流量的上界。在 GPU 上真正决定 B_r 怎么选的，是 tensor core 的利用率和寄存器压力，而 DRAM 流量反倒是次要的。

再把同样的思路用到朴素实现上，看看三种字节模型各自能解释多少耗时。

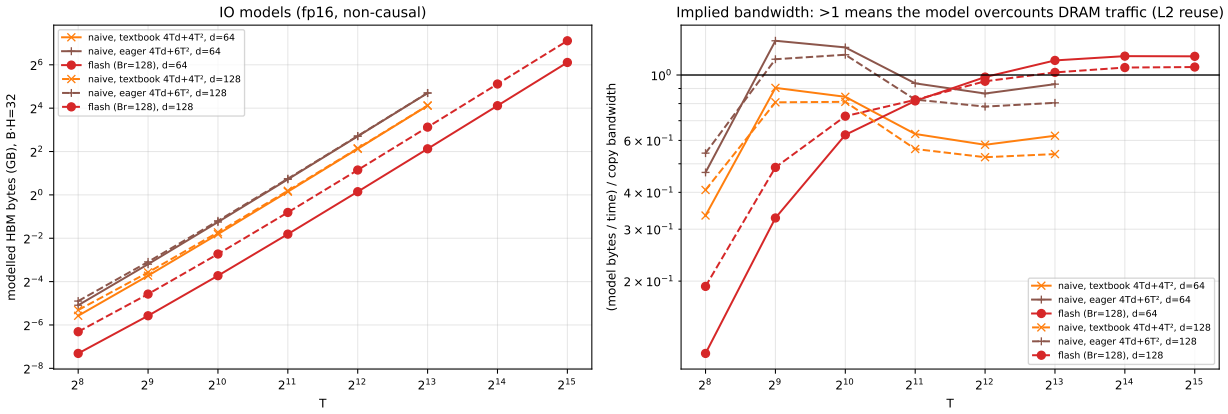


图 10: 左：三种字节模型（fp16，非因果，$B \cdot H = 32$）；右：模型字节 ÷ 实测时间 ÷ copy 带宽。大于 1 表示模型高估了 DRAM 流量。

图 10 把 CPU 部分的 IO 模型直接放到 GPU 时间上对照，可以看到三件事。

朴素实现几乎就是一个以 DRAM 带宽运行的访存程序。如果按教科书的 $4Td + 4T^2$ 计算字节，$T \geq 2048$ 时隐含带宽只有 copy 带宽的 0.55–0.62，模型显然漏算了流量。eager 实现实际的访存是 $4Td + 6T^2$，多出的是 `*scale` 的一读一写。按这个数算，$d = 64$、$T = 8192$ 时为 25.90 GB ÷ 18.28

ms = 1.417 TB/s，是 copy 带宽的 93%。$T = 512$–1024 时 eager 曲线超过 1，因为整个分数张量放得进 96 MB 的 L2（$T = 1024$ 时 $32 \times 1024^2 \times 2 = 67$ MB），读写不必到 DRAM。

Triton（$B_r = 128$）在 $T \geq 4096$ 时比值也超过 1。$T = 16384$ 时为 17.31 GB ÷ 9.818 ms = 1.76 TB/s，是 copy 带宽的 1.16 倍，同样说明 L2 至少吸收了 14% 的 K/V 重读。

字节比和时间比不必相等。模型给出的朴素与 flash 的字节比约为 $2B_r/d \cdot \frac{6}{4} = 6$（$d = 64$），实测时间比是 7.15。flash 是计算受限的，它的时间由 FLOPs 决定，所以时间比本来就不该等于字节比。

4.6 Roofline：一层 Llama-2-7B 的 prefill 与 decode

官方第 3 部分问的是 MHA 的算术强度。要把这个问题落到实测上，先得测出 roofline 的两个参数，再把真实模型的算子放上去，看它们是受带宽限制还是受算力限制。

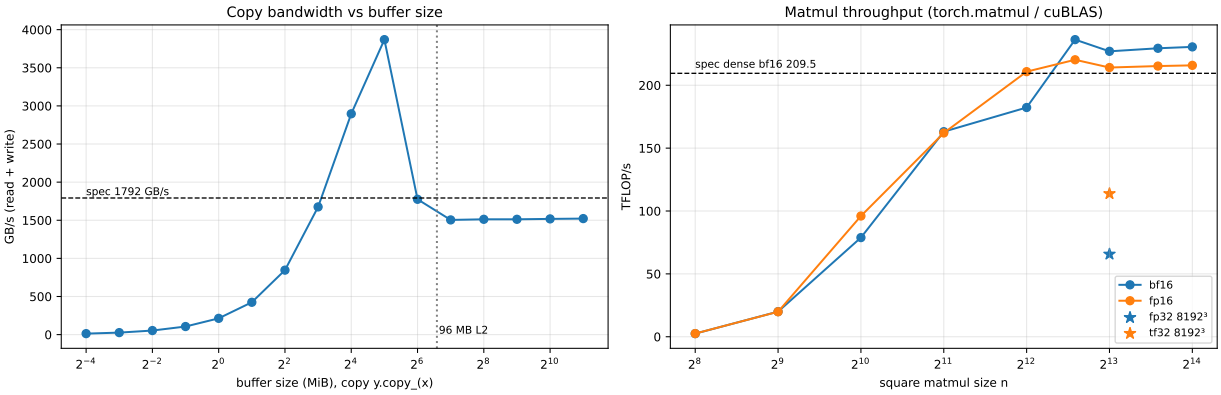


图 11: 左：copy 带宽随缓冲大小的变化（32 MiB 处 3.87 TB/s 是 L2 带宽，≥ 128 MiB 后收敛到 1.50–1.52 TB/s）；右：方阵乘吞吐（`roofline.jsonl`）。

图 11 左图是带宽。缓冲 ≤ 32 MiB 时两个缓冲共 64 MiB，放得进 96 MB 的 L2，copy 带宽达到 3869 GB/s；64 MiB 时开始下降；≥ 128 MiB 后稳定在 1504–1522 GB/s，是规格 1792 GB/s 的 85%。只读求和是 1626 GB/s。右图是算力。矩阵乘在 $n \leq 1024$ 时受启动开销限制，$n = 256$ 只有 2.5 TFLOP/s，单次 13.5 µs；$n = 6144$ 时达到峰值 236.3 TFLOP/s。fp16（fp32 累加）稍低，为 220.3；fp32 不用张量核，只有 65.7；TF32 是 113.8。

有了这两个参数，就可以把一层 Llama-2-7B 的四个算子放到 roofline 上。

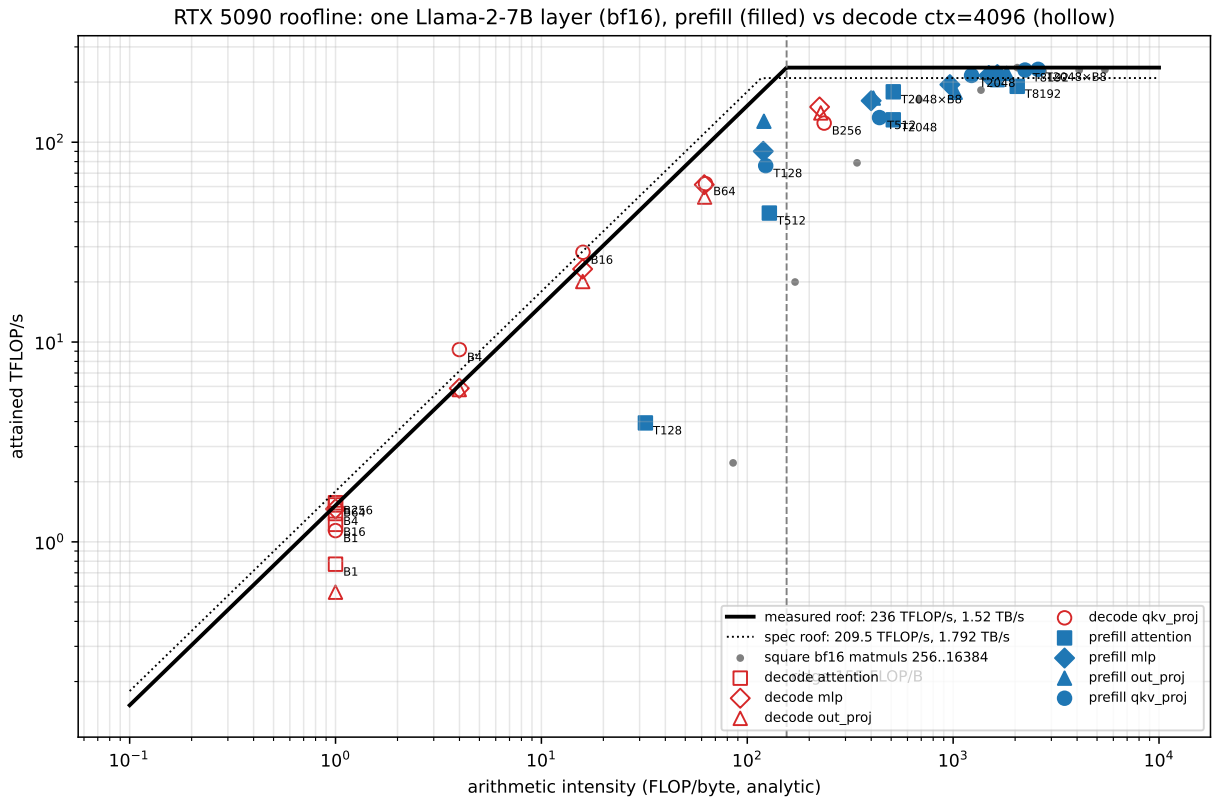


图 12: RTX 5090 roofline。横坐标为 `mlsys_calc` 给出的解析强度，纵坐标为实测吞吐；实心点为 prefill，空心点为 decode（上下文 4096）。

表 4: 一层 Llama-2-7B 各算子（bf16；eff 为 roofline 时间与实测时间之比，roofline 用实测峰值）

形状	算子	强度 (FLOP/B)	ms	TFLOP/s	GB/s	eff
decode $B = 1$	QKV 投影	1.00	0.0884	1.14	1139	0.75
	attention	1.00	0.0868	0.77	773	0.51
	MLP	1.00	0.1851	1.46	1463	0.96
decode $B = 64$	QKV 投影	62.7	0.1039	62.0	989	0.65
	attention	1.00	2.806	1.53	1531	1.01
decode $B = 256$	QKV 投影	236.3	0.2067	124.7	528	0.53
	attention	1.00	10.95	1.57	1570	1.03
prefill $T = 128$	attention	32	0.0341	3.94	123	0.08
prefill $T = 8192$	QKV 投影	2234	3.589	229.8	103	0.97
	attention	2048	2.887	190.5	93	0.81
	MLP	1491	10.27	215.8	145	0.91

图 12 and 表 4 给出了第 3 部分的实测答案，可以分成三类算子来看。

decode 阶段的 attention 强度恒为 1 FLOP/B，与 batch 无关。原因是每个序列都有自己的 KV cache，加大 batch 只会成比例地增加要读的字节，FLOPs 和字节同步增长。这与 CPU 部分 `decode_attention_intensity` 的公式 $I = 2(H/H_{kv})/b$ 完全一致（MHA、$b = 2$ 时为 1）。$B \geq 64$ 时它跑到 1.53–1.57 TB/s，略高于 copy 带宽，因为 attention 几乎只读不写，而只读带宽是 1.63 TB/s。

decode 阶段的投影与 MLP 强度约等于 B。这时权重字节占主导，$I \approx \frac{2BNK}{2NK} = B$。$B = 1$ 时 MLP 读 270 MB 权重，达到 1463 GB/s（eff 0.96）。$B = 256$ 时 QKV 投影的强度 236 已越过 ridge

（155），变成计算受限，但只达到 124.7 TFLOP/s，因为 $M = 256$ 的瘦矩阵乘 tile 利用率差。

prefill 阶段的四个算子都在 ridge 右侧，是计算受限的。$T = 8192$ 时 QKV 投影达到 229.8 TFLOP/s（eff 0.97）。唯一的例外是 $T = 128$ 的 attention，只有 3.94 TFLOP/s（eff 0.08）：这是一个 34 µs 的 kernel，时间几乎全花在启动上。

表中还有一个看似违反 roofline 的点：decode $B = 4$ 的 QKV 投影 eff 为 1.51，超过了 1。它的权重是 $4096 \times 12288 \times 2 = 100.7$ MB，与 96 MB 的 L2 相当，重复测量时有一部分权重留在了 L2 里。真实推理中各层权重轮流使用，不会有这种 L2 命中，所以小 batch 的投影应该按 eff ≈ 0.75–0.96 理解。

知道了每个算子的效率，下一个问题是：一层的时间主要花在哪里？

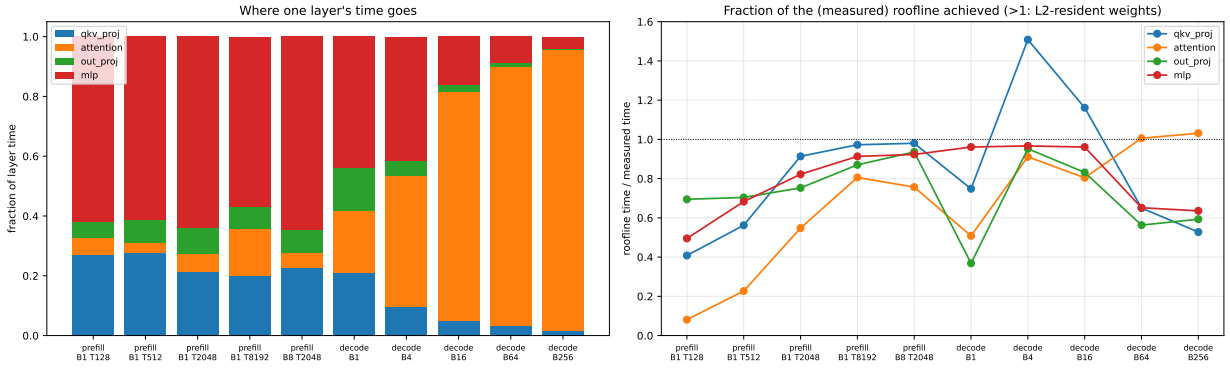


图 13: 左：一层时间在四个算子间的分配；右：各算子达到的 roofline 比例。

图 13 显示，答案随场景完全不同。decode $B = 1$ 时一层耗时 0.420 ms，MLP 占 44%，因为它的权重最多。decode $B = 256$ 时 attention 占 94%（10.95 / 11.67 ms）：投影的权重读取被 256 个序列摊薄了，KV cache 的读取却没有。所以长上下文、大 batch 的 decode 需要的是 GQA/MQA 或 KV 量化，更快的矩阵乘帮不上忙。prefill $T = 8192$ 时情况又反过来，attention 只占 16%（2.887 / 18.08 ms），MLP 占 57%。最后做一个粗略估算：按 decode $B = 1$ 一层 0.420 ms，32 层约 13.4 ms/token（不含 LM head），对应约 74 token/s；而纯权重读取的下界是 13.5 GB ÷ 1.522 TB/s = 8.9 ms。

4.7 Depthwise 卷积

卷积这部分要回答两个问题：手写 kernel 能否打败 cuDNN，以及在 GPU 上什么时候该改用 FFT。先看三个 kernel 与两条基线在不同核长下的耗时。

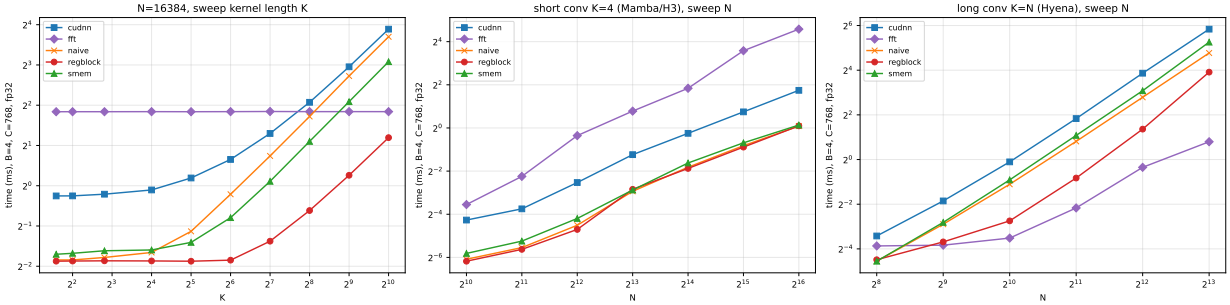


图 14: Depthwise 因果卷积（$B = 4$，$C = 768$，fp32；`dwconv.jsonl`）。左：$N = 16384$ 扫 K；中：$K = 4$ 扫 N；右：$K = N$ 扫 N。

表 5: Depthwise 卷积耗时（ms，$B = 4, C = 768$，fp32；每行最快者加粗）

设置	naive	smem	regblock	cuDNN	FFT
$N = 16384, K = 4$	0.2785	0.3124	**0.2735**	0.8396	3.577
$N = 16384, K = 64$	0.8633	0.5770	**0.2776**	1.573	3.581
$N = 16384, K = 1024$	13.00	8.462	**2.289**	14.82	3.581
$K = N = 512$	0.1343	0.1423	0.0776	0.2768	**0.0701**
$K = N = 4096$	6.900	8.463	2.569	14.57	**0.7851**
$K = N = 8192$	27.28	38.20	15.07	57.18	**1.737**

正确性方面，三个 kernel 与 fp64 cuDNN 参考的最大误差随 K 增长，因为累加项变多了：$K = 4$ 时 8.0e-7，$K = 1024$ 时 7.9e-6，$K = 4096$ 时 1.4e-5；FFT 约 2.5e-6。

图 14 and 表 5 中可以按核长分成两种情况。

短卷积（$K \le 32$）是纯粹的访存问题。$K = 4$ 时三个 kernel 都在 0.27–0.31 ms，regblock 达到 1472 GB/s，即 copy 带宽的 97%。cuDNN 慢 3.07 倍（480 GB/s），它对 depthwise 1D 没有专门的路径。$N \le 4096$ 时数据（$4 \times 768 \times 4096 \times 4 = 50$ MB）能放进 L2，带宽可以超过 DRAM（$N = 4096$ 时 2624 GB/s）。

长卷积（$K \ge 128$）变成了计算问题。$K = 1024$ 时 regblock 耗时 2.289 ms，等效 45.0 TFLOP/s，是 naive 的 5.7 倍、smem 的 3.7 倍、cuDNN 的 6.5 倍。

与 FFT 的交叉点也随之移动。$N = 16384$ 时 FFT 恒为 3.58 ms，regblock 在 $K = 1024$ 时仍快 1.56 倍，线性外推的交叉点约在 $K \approx 1600$。$K = N$ 时 FFT 从 $N = 512$ 起就更快（0.0701 对 0.0776 ms），$N = 8192$ 时快 8.7 倍。CPU 上 $N = 16384$ 的交叉点是 $K \approx 16$–64，GPU 上右移了 25–100 倍。原因是 GPU 的 FMA 吞吐与带宽之比远高于 CPU，而 FFT 受访存限制，享受不到多出来的算力。

为什么 regblock 比 smem 快这么多？把耗时换算成吞吐，就能看出两者分别撞上了哪一堵墙。

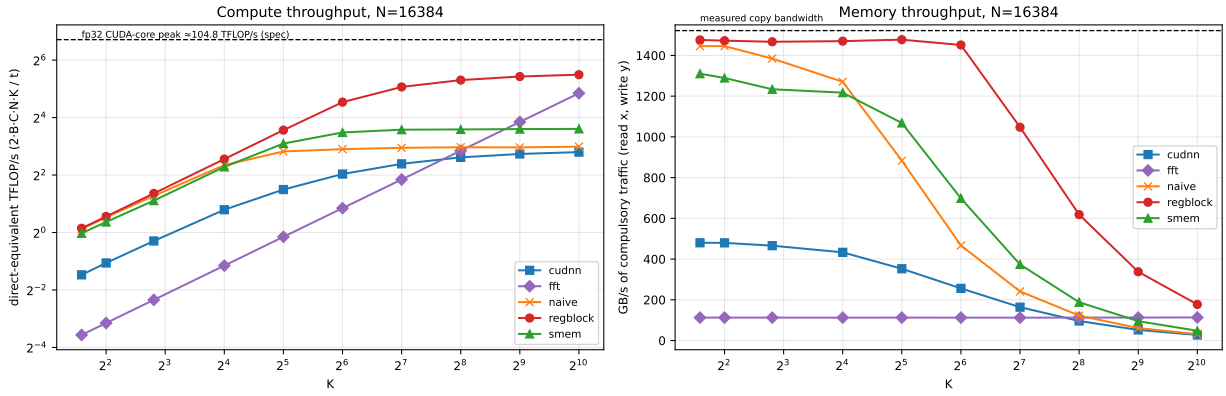


图 15: $N = 16384$ 时的计算吞吐（按直接卷积的 FLOPs 折算）与必需访存吞吐（读 x、写 y）。

图 15 左图中，smem kernel 在 $K \ge 128$ 后停在 12.0–12.2 TFLOP/s，naive 停在 7.7–7.9 TFLOP/s。smem 的平台可以用共享内存带宽解释。每个 FMA（2 FLOP）要 2 次 4 字节的共享内存读，即 4 B/FLOP；共享内存带宽约为 170 SM $\times$ 32 bank $\times$ 4 B $\times$ 2.6 GHz $\approx$ 56.6 TB/s，所以上限约 14 TFLOP/s，实测 12.2 是它的 86%。regblock 每 64 个 FMA 只要 23 次读（h 的 8 次广播加 x 的 15 次），即 0.72 B/FLOP，共享内存给出的上限升到约 79 TFLOP/s。实测 45.0 TFLOP/s 已接近 fp32 CUDA core 规格峰值（104.8）的一半，瓶颈回到了 FMA 单元。这就是寄存器分块要做的全部事情：把共享内存读取次数从 $O(RK)$ 降到 $O(K \cdot 3R/R)$，让瓶颈从片上存储回到计算单元。

4.8 训练：MQAR 与吞吐

前面的 kernel 都是单独测的。训练实验要回答的是另外两个问题：一是第 2 部分手写的两层结构能否被梯度下降学出来，二是这些 kernel 放进真实训练里能带来多少端到端收益。

MQAR：先找一个学得会的设置。 我第一次尝试的设置是 $n_k = 256$、$P = 32$、lr 10^{-3}、wd 0.1，跑了 4000 步，准确率停在约 5%，完全没学会。于是做了两轮扫描（图 16，`train_mqar_sweep*.jsonl`），eval 准确率在 16×512 条新序列上计算，结果如下：

表 6: MQAR 扫描（2 层，bf16 + SDPA）

n_k	P	lr	wd	步数	eval 准确率
64	4	10^{-3}	0	6000	**1.000**
64	8	10^{-3}	0	6000	**1.000**
64	8	3×10^{-3}	0.1	6000	0.364
64	8	3×10^{-3}	0	2000	0.361
64	16	10^{-3}	0.1	6000	0.253
64	16	3×10^{-3}	0	2000	0.250
128	16	10^{-3}	0.1	6000	0.231
256	32	3×10^{-3}	0	2000	0.101
256	32	10^{-3}	0.1	4000	约 0.05

扫描结果有两条规律。其一，学会是「全有或全无」的：准确率要么是 1.000，要么卡在一个明显低于 1 的平台上，没有中间状态。其二，学习率比步数更关键。$P = 8$ 时，lr 3×10^{-3} 跑满 6000 步仍停在 0.36，而 lr 10^{-3} 在第 1000 步左右就跳到了 1.0。$P = 16$ 在 6000 步内两种学习率都没有跳出平台。

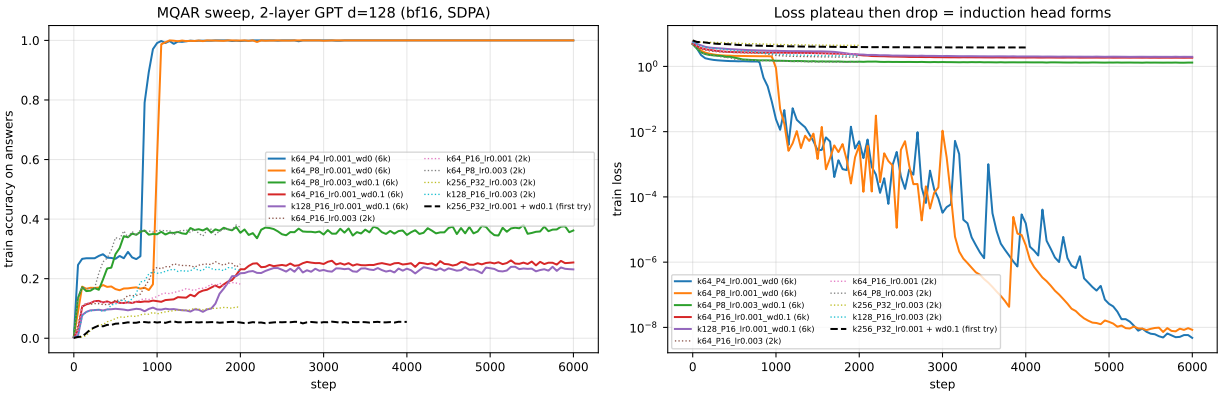


图 16: MQAR 扫描的训练准确率与 loss。实线为 6000 步一轮，点线为 2000 步一轮，黑虚线为第一次尝试。学会的设置都是先停在平台上，再在几十步内跳到 100%。

MQAR 对照实验。 最终选定的设置是 $n_k = 64$、$P = 8$（$T = 32$）、lr 10^{-3}、wd 0。每种配置跑 6000 步，使用同一个随机种子（`train_mqar.jsonl`，图 17）：

表 7: MQAR 对照（$n_k = 64$，$P = 8$，6000 步，batch 256）

配置	参数量	eval 准确率	首次 acc≥0.99	训练时间 (s)	tokens/s
2 层，bf16 + SDPA	413k	1.000	第 1100 步	63.5	0.77 M
1 层，bf16 + SDPA	215k	0.172	—	44.8	1.10 M
2 层，bf16 + Triton（本文）	413k	1.000	第 1100 步	75.1	0.65 M
2 层，fp32 + 朴素	413k	1.000	第 1100 步	63.2	0.78 M

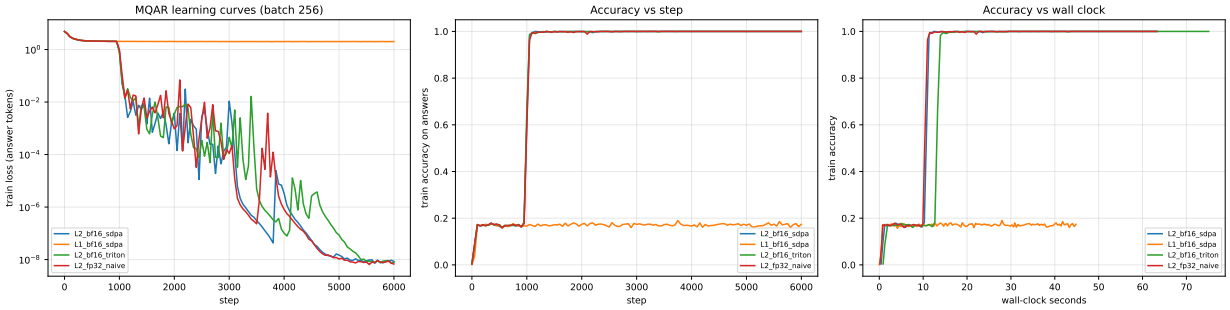


图 17: MQAR 学习曲线（左按步数，右按墙钟时间）。三个 2 层模型的曲线在第 1000 步前几乎重合，都先停在与 1 层模型相同的 0.17 平台，然后同时跳到 100%。

这组对照说明了三件事。

1 层学不会，2 层在第 1000–1100 步突然学会。前 1000 步里，2 层模型的准确率和 1 层模型一样停在约 0.17；然后在 100 步之内跳到 1.0，loss 从 2.0 掉到 10^{-2} 以下。0.17 略高于随机猜的 $1/P = 0.125$。我的理解是，只靠一层 attention，模型能学会「从上下文里出现过的 value 中挑一个」，但无法把查询和对应的 key 匹配起来；高出 $1/P$ 的部分可能来自排除已经回答过的 value，这一点没有验证。匹配需要两步：先让每个 value 位置通过上一 token 头记住它前面的 key，再让查询去匹配这个 key。这正是第 4.1 节 的手写构造和 induction head[11] 的结构，至少需要两层才能实现。

三种 attention 实现学到了同一个结果。bf16 加 SDPA、bf16 加本文 Triton kernel（前向和反向）、fp32 加朴素实现，都在第 1100 步首次达到 0.99，最终都是 1.000。这是对 Triton 反向在真实训练中的端到端校验：如果 dQ、dK、dV 有错，学习曲线不可能与 SDPA 重合。跳变之后 loss 曲线有些差别，Triton 在 10^{-4}–10^{-6} 区间下降稍慢，这来自 bf16 的舍入路径不同，不影响准确率。

在这个规模下，实现选择不影响速度。$T = 32$、$d = 128$ 的模型每步只要约 10 ms，几乎全是 kernel 启动开销，所以 fp32 朴素实现和 bf16 SDPA 一样快（0.78 对 0.77 M tokens/s）。Triton 反而慢 18%，因为它的反向要额外启动一个计算 δ 的 torch kernel 和两个 Triton kernel，还要经过 `autograd.Function` 的 Python 开销。

吞吐与 MFU。 小模型看不出 kernel 的差别，所以吞吐实验换到 GPT-2-small 形状：110.2M 参数（不含位置编码），$T = 1024$，$B = 8$，每步 8192 tokens。每种配置先预热 3 步，再取 10 步（`train_throughput.jsonl`，图 18 and 表 8）。

表 8: GPT-2-small 形状的训练吞吐（$T = 1024$，$B = 8$）。MFU 的分母为实测 BF16 峰值 236.3 TFLOP/s

精度	attention	每步 (ms)	k tokens/s	MFU（$6N$）	MFU（$+12LTd$）	峰值显存 (GB)
fp32	朴素	197.3	41.5	0.116	0.136	15.38
fp32	SDPA	137.5	59.6	0.167	0.195	10.53
TF32	朴素	158.0	51.9	0.145	0.170	15.38
TF32	SDPA	99.6	82.2	0.230	0.270	10.54
bf16	朴素	126.4	64.8	0.181	0.213	15.37
bf16	SDPA	**51.3**	**159.6**	**0.447**	**0.523**	8.27
bf16	Triton（本文）	52.5	156.0	0.437	0.512	8.25

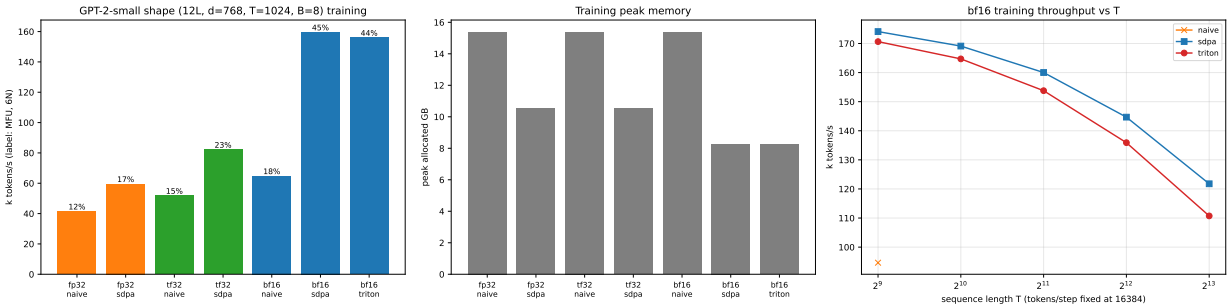


图 18: 左：吞吐与 MFU（柱上数字）；中：峰值显存；右：固定每步 16384 tokens 时吞吐随 T 的变化（朴素实现从 $T = 1024$、$B = 16$ 起 OOM）。

从这组数据里可以得到以下几点。

bf16 autocast 和 flash attention 要一起用才有效果。只开 bf16、保留朴素 attention，只快 1.56 倍（41.5k 到 64.8k）；只换 SDPA、保持 fp32，只快 1.44 倍；两者一起才快 3.84 倍（159.6k）。原因是朴素 attention 的 T^2 分数张量是访存受限的。它有 $B \cdot H \cdot T^2 = 8 \times 12 \times 1024^2$ 个元素，每层还要保存 softmax 的输入和输出供反向使用。bf16 只能把这部分字节减半，而 tensor core 的算力只有在矩阵乘部分才用得上。

显存上，SDPA 把峰值从 15.4 GB 降到 8.3 GB（bf16），省下的正是每层保存的 T^2 张量。每步 16384 tokens（$T = 1024$、$B = 16$）时，bf16 朴素实现在 32 GB 显存上已经 OOM。

本文的 Triton kernel 达到了 SDPA 的 97.7%（156.0k 对 159.6k）。前向基本持平，差距来自反向，这与图 7 中反向只有 PyTorch FA2 的 80% 一致。

MFU 方面，bf16 加 SDPA 达到 0.447（按 $6N$），计入 attention 后为 0.523。剩下约一半的时间花在 LayerNorm、GELU、残差、交叉熵（词表 32768）和 AdamW 这类访存受限的操作上。

序列长度的影响要小心解读。固定每步 16384 tokens 时，SDPA 的吞吐从 $T = 512$ 的 174.1k 降到 $T = 8192$ 的 121.8k，因为 attention 的 FLOPs 随 T 线性增长。按 $6N$ 计的 MFU 随之降到 0.34，但按 PaLM 公式计入 attention 后升到 0.81。这个 0.81 偏高：PaLM 的 $12LTd$ 计的是非因果的全部 attention FLOPs，而因果 kernel 只算一半。按 $6LTd$ 修正后，MFU 在 $T = 512$–8192 之间稳定在 0.51–0.57。另外，Triton 在 $T = 8192$ 时为 110.7k（SDPA 的 91%），序列越长，反向的差距越明显。

4.9 手写 associative recall 在 GPU 上的复核

最后一个实验很小，目的是确认图 3 的手工构造不依赖设备。我把 CPU 部分的手工权重模型原样放到 GPU 上（`train_recall.jsonl`），跑 4 种键值对数乘 6 个 β，共 24 组，每组 5000 条序列，随机种子与 CPU 实验相同。GPU 与 CPU 的准确率逐格相同，预测不一致的条数为 0。例如 32

对、$\beta = 4$ 时两者都是 0.9988，$\beta = 1$ 时都是 0.2268。每组 5000 条序列在 GPU 上前向约 0.7–4 ms（第一次调用含 CUDA 初始化，为 193 ms）。这说明该构造与设备无关，fp32 下 $\beta = 30$ 的 softmax 也没有数值问题。

5 与理论模型的对照

前面的结果里多次出现「可以用模型解释到 93%」这样的说法。这一节把背后的推导集中写出来，目的是让读者拿到一个新形状时，不用跑实验也能估出大致的数量级。

5.1 Attention 的 FLOPs、字节与强度

单头前向的 FLOPs 是：$QK^\top$ 与 PV 各 $2T^2d$，共 $4T^2d$，因果时有用部分减半。三种实现的字节数（fp16，每元素 2 B）和强度如下：

$$\text{朴素（教科书）}: 2(4Td + 4T^2) \qquad I = \frac{4T^2d}{8T^2} = \frac{d}{2}, \tag{2}$$

$$\text{朴素（eager）}: 2(4Td + 6T^2) \qquad I = \frac{4T^2d}{12T^2} = \frac{d}{3}, \tag{3}$$

$$\text{Flash}: 2\left(2Td + \frac{T}{B_r} \cdot 2Td\right) \qquad I \approx \frac{4T^2d}{4T^2d/B_r} = B_r. \tag{4}$$

朴素实现在 $d = 64$ 时强度为 21.3 FLOP/B，远低于 ridge 155。按 roofline 预测吞吐为 $1.522\ \text{TB/s} \times 21.3 = 32.5$ TFLOP/s，实测 30.1（$T = 8192$），误差 7%。$d = 128$ 时预测 65.0、实测 51.7。因果 eager 实现多了 `masked_fill` 的一读一写（共 $8T^2$），有用 FLOPs 却减半，所以 $I = d/8 = 8$，预测 12.2、实测 8.7 TFLOP/s。

式 (4) 给出一个在面试中很好用的结论：FlashAttention 前向的 DRAM 强度等于 Q 块的行数 B_r（fp16 时）。$B_r = 128$ 时 $I = 128 < 155$，按模型它应该略低于计算屋顶，即 $1.522 \times 128 = 195$ TFLOP/s。但实测是 215–224 TFLOP/s，超过了这个值，这又一次说明 L2 让实际的 DRAM 字节少于模型。论文中的 $\Theta(T^2d^2/M)$ 就是把 $B_r \propto M/d$ 代入 T^2d/B_r 得到的。在 sm_120 上 $M \approx 99$ KB，而 B_r 被寄存器容量限制在 128 左右，因此在 $d = 64$–128 时流量仍然是 $\Theta(T^2)$，只是常数比朴素实现小 $\frac{3B_r}{d}$ 倍（按 eager 计数）。

5.2 为什么 FlashAttention 快：三个因素的分解

以 $d = 64$、$T = 8192$、非因果为例，Triton 耗时 2.558 ms，朴素实现 18.28 ms，比值 7.15。这个比值可以分三步算出来：

1. 朴素实现受访存限制，时间约为 $25.90\ \text{GB}/1.522\ \text{TB/s} = 17.0$ ms，占实测 18.28 ms 的 93%；

2. flash 受计算限制，时间约为 $4 \cdot 32 \cdot 8192^2 \cdot 64/236.3$ TFLOP/s $= 2.33$ ms，占实测 2.558 ms 的 91%；

3. 两者相除，加速比约为 $\frac{\text{bytes}_{\text{eager}}/BW}{\text{FLOPs}/\text{peak}} = \frac{6T^2 \cdot 2/BW}{4T^2d/\text{peak}} = \frac{3 \cdot \text{ridge}}{d} = \frac{3 \times 155.3}{64} = 7.3$，与实测 7.15 吻合。

这个式子还能解释另外两个现象。$d = 128$ 时加速比更小：预测 $\frac{3 \times 155.3}{128} = 3.6$，实测 21.26/5.617=3.8。因果时加速比更大：朴素实现多读写一次且仍然算满 T^2，而 flash 跳过了一半的块。

5.3 Roofline 中的线性层

线性层的强度由 $\texttt{gemm_intensity}(M, N, K) = \frac{2MNK}{2(MK+KN+MN)}$ 给出。decode 时 $M = B \ll N, K$，所以 $I \approx \frac{2BNK}{2NK} = B$。ridge 为 155，意味着 batch 要超过约 155，投影才会变成计算受限。这与实测中 $B = 256$ 的 QKV 投影（$I = 236$）首次越过 ridge 一致。这和 `mlsys_calc.critical_batch`

的思路相同：临界 batch ≈ ridge × 每元素字节 / 2。decode attention 的强度是 $\frac{4Hd_hT}{2\cdot 2Hd_hT}=1$，与 batch 和上下文长度都无关。GQA-8[13]（Llama-3-8B）把它提高到 4，MQA[14] 提高到 32，但都仍低于 ridge。所以 GQA 的收益在于读更少的字节，attention 并没有因此变成计算受限。

5.4 卷积

直接卷积的 FLOPs 是 $2BCNK$，必需字节是 $4(2BCN+CK)$，强度约为 $K/4$。$K=4$、$N=16384$ 时 $I=1$，受访存限制。必需字节为 $4(2\cdot 4\cdot 768\cdot 16384+768\cdot 4)=402.7$ MB，按 copy 带宽预测 0.265 ms，实测 regblock 0.2735 ms（1472 GB/s，97%）。$K=1024$ 时 $I=256$，按 DRAM 算是计算受限，但真正的瓶颈是片上的共享内存带宽，推导见第 4.7 节。FFT 卷积的 FLOPs 只有 $O(N\log N)$，但要对长度 $2N$ 的复数数组做多遍读写，所以受访存限制。GPU 的 ridge 比 CPU 高得多，直接卷积多出来的 FLOPs 在 GPU 上很便宜，于是交叉点 K^* 大幅右移。

6 面试专题

20 秒电梯陈述 我在 RTX 5090 上用 Triton 写了 FlashAttention-2 的前向和反向。前向在 $T=16$k 时跑到 224 TFLOP/s，是实测矩阵乘峰值的 95%，和 PyTorch 自带的 FA2 持平；比朴素实现快 7 倍，因果时快 22 倍；显存从 8.6 GB 降到 35 MB。我用 roofline 把加速比拆成了一个公式：朴素实现以 93% 的 DRAM 带宽运行，flash 以 91% 的算力运行，所以加速比约等于 $3\times\text{ridge}/d$。我还发现教科书的 IO 模型在 GPU 上最多会把 DRAM 流量高估 4 倍，因为 K/V 的重读命中了 96 MB 的 L2。

STAR 故事 情境（S）：课程作业只要求 numpy 版的 FlashAttention。它在 CPU 上比朴素实现还慢（$T=1024$ 时 44.6 对 22.9 ms），IO 模型也只说它的流量少一半，很难解释为什么工业界都在用它。任务（T）：在真实 GPU 上量化它到底快多少、为什么快，并检验 IO 模型。行动（A）：用 Triton 写前向和反向，用 fp32 参考实现验证（35 个 GPU 测试）；扫描 T、d、因果、dtype 和 $B_r\times B_c$；实测 copy 带宽和矩阵乘峰值，把每个实现的时间换算成隐含带宽和算力利用率。结果（R）：前向 224.0 TFLOP/s，与 SDPA flash（217.9）持平；朴素实现在 $T=16384$ 时 OOM，因为两个 fp16 $T\times T$ 张量要 34 GB；用 $3\cdot\text{ridge}/d=7.3$ 预测出了实测 7.15 倍的加速；块扫描中 6.57 TB/s 的隐含带宽证明了 L2 复用。反向只有 FA2 的 80%，我能定位到原因：两个 kernel 的结构多做了 2 个矩阵乘。

面试追问：FlashAttention 到底省了什么？流量是不是降了一个阶？

首先省的是显存。它不物化 $T\times T$ 的 S 和 P，额外显存从 $O(T^2)$ 降到 $O(T)$，$T=8192$ 时从 8.62 GB 降到 34.6 MB。流量的阶其实没变：理论值是 $\Theta(T^2d^2/M)$，但 $d=64$–128 时 B_r 被 SRAM 和寄存器封顶，所以仍是 $\Theta(T^2)$，只是常数小了 $3B_r/d$ 倍。时间上的收益来自把一个访存受限的程序变成了计算受限的程序，加速比约为 $3\cdot\text{ridge}/d$。

常见错答：「FlashAttention 把 attention 从 $O(T^2)$ 变成了 $O(T)$。」实际上计算量仍是 $4T^2d$，也没有做任何近似。

面试追问：FlashAttention 前向的算术强度是多少？

fp16 下约等于 Q 块的行数 B_r。每个 Q 块读一遍整段 K/V（$4Td$ 字节），做 $4B_rTd$ 次 FLOP。$B_r=128$ 时 $I\approx 128$，接近 5090 的 ridge 155。所以 B_r 不能太小（$B_r=16$ 时吞吐只有一半），也不能太大（寄存器溢出，$B_r=256$ 时下降）。

常见错答：「强度和 T 成正比，所以序列越长越计算受限。」其实每个 Q 块对每个 K/V 元素只用一次，强度与 T 无关。

面试追问：为什么朴素 attention 在 GPU 上只有 30 TFLOP/s？

因为它受访存限制。eager 实现中，matmul 写 S、scale 读写、softmax 读写、matmul 读 P，一共 $6T^2$ 次 fp16 访问，强度只有 $d/3 \approx 21$ FLOP/B，远低于 ridge 155。按 1.522 TB/s 预测是 32.5 TFLOP/s，实测 30.1。

常见错答：「因为 softmax 很慢。」softmax 只占其中两次 T^2 访问，换成任何逐元素操作都一样慢；真正的问题是物化了 $T \times T$ 的中间结果。

面试追问：online softmax 为什么正确？反向为什么只存 lse？

它维护 $m' = \max(m, s)$，$\ell' = \ell e^{m-m'} + \sum e^{s-m'}$，$\text{acc}' = \text{acc}\, e^{m-m'} + e^{s-m'}V$，任意时刻 acc/ℓ 都等于已见过的 key 上的精确 softmax 加权和。反向用 $P = \exp(S - \text{lse})$ 重算，只需要一个标量 $\text{lse} = m + \log \ell$，比存 m 和 ℓ 两个量省显存，也省一次除法。本文用以 2 为底的版本，因为 `exp2` 是一条 SFU 指令。

常见错答：「online softmax 是近似算法。」它是精确的重排，误差只来自浮点舍入；fp16 下为 6.6e-5，与 PyTorch FA2 相同。

面试追问：decode 阶段加大 batch 能让 attention 变成计算受限吗？

不能。decode attention 的强度是 $2(H/H_{kv})/b$，MHA 时为 1，与 batch 和上下文都无关，因为每个序列有自己的 KV cache。实测 B 从 1 到 256，强度一直是 1.00；$B = 256$ 时 attention 占一层时间的 94%。加大 batch 只对线性层有用（强度约等于 B），要到 $B \approx 155$ 才越过 ridge。

常见错答：「batch 越大 GPU 利用率越高，所以 decode 都应该开大 batch。」这对 attention 无效。要容纳更大的 batch，得靠 GQA/MQA、KV 量化或 PagedAttention。

面试追问：IO 模型说 flash 流量是朴素的 1/2（或 1/6），为什么 GPU 上测不到这个比例？

有三个原因。eager 朴素实现的 T^2 访问是 6 次而不是 4 次；GPU kernel 的 $B_r = 128$，大于 CPU 模型里的 $B_r = d = 64$；K/V 重读大量命中 L2，块扫描中按模型算出的隐含带宽高达 6.57 TB/s，是 DRAM 的 4.3 倍。此外 flash 是计算受限的，时间比本来就不等于字节比。

常见错答：「模型错了。」模型给的是 DRAM 流量的上界。在只有一级 SRAM、没有 L2 的假设下，它是对的。

面试追问：depthwise 卷积什么时候用 FFT？

要看硬件。直接卷积是 $O(NK)$，强度约 $K/4$；FFT 是 $O(N \log N)$，但受访存限制。GPU 上 $N = 16384$ 时交叉点约在 $K \approx 1600$：$K = 1024$ 时寄存器分块 kernel 用 2.29 ms，仍快于 FFT 的 3.58 ms。$K = N$ 时，$N \geq 512$ 就该用 FFT。CPU 上交叉点则是 $K \approx 16$–64。所以 Mamba/H3 的短卷积（$K = 4$）直接写，Hyena 的长卷积（$K = N$）用 FFT。

常见错答：「$K > \log N$ 就用 FFT。」这是只数 FLOPs 得出的结论；在 GPU 上直接卷积的 FLOPs 很便宜，FFT 的访存反而更贵。

面试追问：共享内存 tiling 为什么在长卷积上只快 1.5 倍？寄存器分块解决了什么？

因为 smem 版撞上了共享内存带宽。它每个 FMA 仍要读 2 次共享内存（4 B/FLOP），而共享内存带宽约 57 TB/s，上限约 14 TFLOP/s，实测 12.2。寄存器分块每 64 个 FMA 只读 23 次（0.72 B/FLOP），上限升到约 79，实测 45.0 TFLOP/s，比 smem 快 3.7 倍。实现时还要处理 stride-8 访

问造成的 8 路 bank 冲突（用填充下标 `i+(i>>5)`），以及输出的合并写。

常见错答：「共享内存和寄存器一样快，搬进共享内存就够了。」共享内存带宽比寄存器低一个数量级，复用度高的 kernel 必须在寄存器层再分一次块。

面试追问：为什么 cuDNN 的 depthwise conv1d 比手写 naive 还慢？

因为它没有为这个形状优化。$K = 4$ 时 cuDNN 用 0.840 ms，naive 只要 0.279 ms。cuDNN 的卷积主要为 2D、通道混合的卷积优化（implicit GEMM），depthwise 1D（groups=C）没有专门路径，还要先用 `F.pad` 复制一次输入，多了一读一写。这也是 Mamba/H3 都自己写 `causal_conv1d` kernel 的原因。

常见错答：「库一定比手写快。」库只对它优化过的形状快。

面试追问：MFU 怎么算？为什么小模型 MFU 低？

MFU $= 6N \cdot \text{tokens/s}/\text{peak}$（`mlsys_calc.mfu`）。$6N$ 是每个 token 前向 $2N$、反向 $4N$ 的 FLOPs，不含 attention 的 $12LTd$；分母用实测 BF16 峰值 236.3 TFLOP/s。小模型（$d = 128$）的矩阵乘太小，启动开销和 LayerNorm、GELU、优化器这类逐元素操作占了主导。GPT-2-small 形状的数字见第 4.8 节。

常见错答：「MFU 低说明 GPU 坏了，或者 kernel 写得差。」MFU 低首先是形状问题。另外，不能不加说明地拿 fp32 训练的 MFU 去和 bf16 峰值比较。

7 局限

下面这些结论是本文的数据支撑不了的，引用时请注意边界。

不能推出的结论

- 没有直接测量 DRAM 字节。容器无权读取性能计数器（ncu 报 `ERR_NVGPUCTRPERM`），所有关于流量的结论都来自「模型字节 ÷ 实测时间」得到的隐含带宽。L2 复用是从隐含带宽超过 DRAM 峰值推断出来的，没有给出命中率。
- 不能推出「Triton kernel 比 PyTorch FA2 更快」。它只在 $T \geq 8192$、$d = 64$ 时快 2–4%，短序列和 $d = 128$ 时更慢，反向只有 80%；而且 PyTorch 的 FA2 未必针对 sm_120 调过优。
- 不能推出 decode 小 batch 投影能超过 roofline。$B = 4$ 的 eff 1.51 来自重复测量时权重留在 L2，真实推理会逐层轮换权重。
- 只测了单卡、单层。没有测通信、整模型的 kernel 融合和 CUDA graph；层内 MLP 的 `silu·` 是单独的逐元素 kernel，没有融合。
- GPU 是共享的。计时在共享锁内独占完成，但 CPU 只有 2 个核，主机侧开销（如 Triton 反向在短序列上约 0.53 ms 的平台）比独占机器大。
- 卷积 kernel 只做了 fp32 前向，没有 bf16、没有反向，也没有像 `causal_conv1d` 那样为 $K \leq 4$ 做专门的向量化路径。
- MQAR 结果只有一个随机种子，学习曲线的转折步数可能随种子变化。任务规模也很小（$P \leq 8$ 学得会，$P = 16$ 在 6000 步内学不会），不能外推到 Zoology 论文中的长序列设置。
- 「计入 attention 的 MFU」按 PaLM 的 $12LTd$ 计算，即非因果的全部 attention FLOPs。因果 kernel

跳过了一半，按 $6LTd$ 计算时，$T = 8192$ 的 0.81 应为 0.57。

8 结论

结论：FlashAttention

Triton FA2 前向在 $T = 16384$、$d = 64$ 时达到 224.0 TFLOP/s（实测峰值的 95%），与 PyTorch FA2 持平，比朴素实现快 7.15 倍（因果 22.2 倍），额外显存从 8.62 GB 降到 34.6 MB。这个加速比可以用 $3 \cdot \text{ridge}/d$ 预测（7.3 对实测 7.15）。

结论：IO 模型

教科书 IO 模型在 GPU 上只是 DRAM 流量的上界：flash 的模型字节对应的隐含带宽最高 6.57 TB/s，是 DRAM 的 4.3 倍，说明 K/V 重读主要由 96 MB L2 承担。朴素 eager 实现则老老实实受 DRAM 限制，$6T^2$ 的模型解释了它 93% 的耗时。

结论：Roofline

5090 实测 1.522 TB/s、236.3 TFLOP/s，ridge 为 155 FLOP/B。decode attention 的强度恒为 1，$B = 256$ 时占一层 94% 的时间，所以大 batch decode 的瓶颈在读 KV cache；prefill 的算子全部计算受限。

结论：卷积

寄存器分块把瓶颈从共享内存带宽挪回了 FMA 单元：$K = 1024$ 时达到 45.0 TFLOP/s，比 cuDNN 快 6.5 倍、比 smem 版快 3.7 倍；$K = 4$ 时达到 copy 带宽的 97%。GPU 上 FFT 的交叉点（$N = 16384$ 时 $K \approx 1600$）比 CPU 右移了 25–100 倍。

结论：训练

MQAR 上 2 层 GPT 在第 1000–1100 步从 0.17 的平台突然跳到 100%，1 层模型 6000 步仍停在 0.172；SDPA、本文 Triton kernel 和 fp32 朴素实现的学习曲线完全重合，这是对 Triton 反向的端到端校验。GPT-2-small 形状下，bf16 加 SDPA 达到 159.6k tokens/s、MFU 0.447，是 fp32 朴素实现的 3.84 倍。

参考文献

[1] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, C. Ré. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. NeurIPS 2022.

[2] T. Dao. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. ICLR 2024.

[3] M. Milakov, N. Gimelshein. Online normalizer calculation for softmax. arXiv:1805.02867, 2018.

[4] M. N. Rabe, C. Staats. Self-attention Does Not Need $O(n^2)$ Memory. arXiv:2112.05682, 2021.

[5] S. Williams, A. Waterman, D. Patterson. Roofline: An Insightful Visual Performance Model for Multicore Architectures. CACM 52(4), 2009.

[6] P. Tillet, H. T. Kung, D. Cox. Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations. MAPL 2019.

[7] D. Y. Fu, T. Dao, K. K. Saab, A. W. Thomas, A. Rudra, C. Ré. Hungry Hungry Hippos: Towards Language Modeling with State Space Models. ICLR 2023.

[8] M. Poli et al. Hyena Hierarchy: Towards Larger Convolutional Language Models. ICML 2023.

[9] A. Gu, T. Dao. Mamba: Linear-Time Sequence Modeling with Selective State Spaces. arXiv:2312.00752, 2023.

[10] S. Arora, S. Eyuboglu, A. Timalsina, I. Johnson, M. Poli, J. Zou, A. Rudra, C. Ré. Zoology: Measuring and Improving Recall in Efficient Language Models. ICLR 2024.

[11] C. Olsson et al. In-context Learning and Induction Heads. Transformer Circuits Thread, 2022.

[12] A. Chowdhery et al. PaLM: Scaling Language Modeling with Pathways. JMLR 24, 2023（附录 B：MFU 定义）.

[13] J. Ainslie et al. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. EMNLP 2023.

[14] N. Shazeer. Fast Transformer Decoding: One Write-Head is All You Need. arXiv:1911.02150, 2019.

[15] T. Dao, D. Haziza, F. Massa, G. Sizov. Flash-Decoding for long-context inference. PyTorch blog, 2023.

[16] H. Touvron et al. Llama 2: Open Foundation and Fine-Tuned Chat Models. arXiv:2307.09288, 2023.